Challenging Questions (Bonus)

1. Explain the Bias–Variance Trade-off in Regression Models

Bias: The error introduced by approximating a real-world, complex problem with a too-simple model. A linear model trying to predict highly non-linear hotel prices will have high bias (it underfits the data).
Variance: The error introduced by a model being too sensitive to tiny fluctuations or noise in the training data. An unconstrained Decision Tree will map every single outlier in the training data, leading to high variance (it overfits the data and fails on new data).
The Trade-off: As you increase model complexity, bias decreases, but variance increases. The goal of machine learning is to find the mathematical "sweet spot" in the middle that minimizes the Total Error (Bias + Variance + Irreducible Noise).

2. When does Kernel Regression outperform Linear Regression?

Linear Regression can only draw a flat, straight plane through data. It fails completely when the relationships in the data are highly non-linear or have complex interactions (e.g., the price of a room spikes only if the month is August and it's a Resort hotel).
Kernel Regression outperforms Linear Regression because it implicitly projects the data into a much higher-dimensional space using the "Kernel Trick." In this higher dimension, tangled, complex data becomes mathematically separable, allowing the model to capture wavy, highly non-linear patterns without you having to manually engineer polynomial features.

3. Compare L1 vs L2 Regularization

Regularization artificially penalizes a model for having weights that are too large, preventing it from overfitting to noise.
When does LASSO (L1) perform better? When you have a massive dataset with hundreds of useless features and you want the model to do feature selection.
When does Ridge (L2) perform better? When your features are highly correlated (multicollinearity) and you want to keep all of them, but smoothly shrink their impact to prevent any one feature from dominating.
Why does LASSO produce sparsity (weights exactly zero)? Think of the penalty mathematically. L2 (Ridge) uses squared weights, which creates a circular constraint space. The loss function usually intersects this circle at a curve, meaning weights get very small, but rarely zero. L1 (LASSO) uses absolute values, which creates a diamond-shaped constraint space with sharp corners. The loss function is highly likely to intersect the diamond exactly at a corner, which forces that feature's weight to become exactly 0.0, dropping it from the model entirely (sparsity).

4. Explain why MAPE is unreliable in some datasets

MAPE (Mean Absolute Percentage Error) calculates the average percentage a prediction is off by. It is mathematically unreliable because its formula requires dividing the error by the actual value.
If your dataset contains actual values of exactly zero (for example, complimentary hotel rooms or point-redemption bookings where adr = 0), the formula divides by zero, resulting in an infinite error or mathematically crashing the model. It is also inherently asymmetric: it penalizes predicting too high much more harshly than predicting too low.

5. Discuss the effect of outliers on regression models

Models that optimize based on Mean Squared Error (MSE), like standard Linear Regression, are drastically manipulated by outliers. Because the error is squared, being off by $10 creates a penalty of 100, but being off by $100 creates a penalty of 10,000. A single extreme outlier (e.g., a $5,000 presidential suite) will act like a magnet, pulling the entire regression line away from the normal data just to minimize that one squared penalty. Models using Absolute Error (MAE) or Huber Loss are far more robust to outliers.

6. Explain the effect of class imbalance on binary metrics. Why is accuracy misleading?

If 95% of hotel bookings check out, and 5% cancel, the data is imbalanced. A "dumb" model that simply guesses "Check-out" for every single customer will achieve 95% Accuracy without learning a single pattern. Accuracy is entirely inflated by the majority class (True Negatives). To understand if the model is actually working, we must look at metrics that focus on the minority class: Precision (when it predicts a cancellation, is it right?) and Recall (did it catch all the actual cancellations?).

7. Explain how the decision boundaries of your models differ fundamentally

Linear/Logistic Models: Draw a rigid, straight line (or flat plane) slicing through the data space.
Decision Trees: Draw a "staircase" boundary. They partition the data by drawing perfectly vertical and horizontal orthogonal lines parallel to the feature axes (e.g., Age > 30, Price < 100).
KNN: Creates a "Voronoi tessellation." The boundary is highly jagged, localized, and irregular, curving tightly around individual clusters of data points.
Kernel SVM: Draws smooth, wavy, organic curves that can wrap around distinct clusters in completely non-linear shapes.

8. Explain the effect of K in KNN

Small K (e.g., K=1): The model looks only at the single closest neighbor. This leads to extremely high variance; it will memorize every piece of noise and outlier, drawing a heavily jagged boundary.
Large K (e.g., K=100): The model averages a massive area. This smooths out noise, lowering variance but increasing bias. If K gets too large, the model just defaults to predicting the majority class of the entire dataset.

9. Overfitting in Decision Trees

Why do they overfit? If unconstrained, a tree will continue to split until every single leaf node contains exactly 1 customer. It effectively memorizes the training data perfectly (100% accuracy) but will fail spectacularly on unseen data.
Why is max depth not enough? You can set a shallow tree (e.g., max_depth=3), but if a specific sequence of 3 questions isolates a single, bizarre outlier customer, the model has still overfit to that specific path.
How does pruning work? Cost Complexity Pruning grows a massive tree, then evaluates the branches from the bottom up. It collapses leaves back into a single node if the "Information Gain" from that split isn't large enough to justify the complexity penalty (alpha).

10. Explain why Tree-based models are good feature selectors

When a tree decides where to split, it iterates through features and calculates the exact mathematical drop in impurity (Gini or Entropy). If a feature is useless, it will yield zero information gain and simply never be used by the tree. In ensembles (like Random Forest), you can calculate "Feature Importance" by summing up the total impurity decrease a specific feature was responsible for across all hundreds of trees, providing a mathematically robust ranking of the most predictive features.

11. Micro vs Macro vs Weighted F1

Macro F1: Calculates the F1 for each class independently, then averages them equally. It treats a tiny class (100 samples) as equally important as a massive class (10,000 samples). Use this when minority classes matter deeply.
Weighted F1: Averages the F1 scores, but weights them by how many samples exist in that class. Why it's misleading: If the majority class has 99% of the data, it dictates 99% of the score, hiding the model's complete failure on the minority class.
Micro F1: Puts all True Positives, False Positives, and False Negatives from all classes into one giant global pool and calculates one F1 score. Why it favors large classes: Because the large classes contribute the vast majority of the raw numbers to the global pool, their performance totally masks the minority class.

12. Multi-label vs Multiclass

Fundamental Difference: Multiclass is mutually exclusive (a pet is a Dog or a Cat or a Bird). Multi-label is non-mutually exclusive (a movie is Action and Sci-Fi and Comedy).
Output Space: Multiclass outputs a single vector where probabilities sum exactly to 1. Multi-label outputs independent probabilities for each class that do not sum to 1.
Loss Functions: Multiclass uses Categorical Cross-Entropy (Softmax). Multi-label uses Binary Cross-Entropy calculated independently for every class (Sigmoid).
Thresholding: Multiclass uses argmax (pick the single highest probability). Multi-label checks if each output passes a threshold (e.g., if Class A > 0.5 AND Class C > 0.5, apply both labels).
Metrics: Multiclass uses F1 and Kappa. Multi-label requires metrics that evaluate partial correctness, like Hamming Loss or Jaccard Index.
Extending KNN/Trees: ML-KNN independently calculates the probability of each label among its neighbors. ML-Trees modify their leaf nodes to output a vector of probabilities and change their split math to maximize purity across all labels simultaneously.

13. Explain Precision–Recall Trade-off

There is a mathematical tension between catching everything and being perfectly accurate.
If you want high Recall (catching every single hotel cancellation), you must lower the model's threshold. The model will flag almost everyone as a cancellation risk. You catch them all, but your Precision drops horribly because you created hundreds of False Positives.
If you want high Precision (you only flag someone if you are 99.9% sure they will cancel), you raise the threshold. Your predictions will be incredibly accurate, but your Recall plummets because you missed dozens of subtle cancellations. You cannot maximize both.

14. Explain ROC vs PR Curve

ROC Curve (TPR vs FPR): Plots the True Positive Rate against the False Positive Rate. Because FPR includes True Negatives in its denominator, ROC curves look highly optimistic and inflated when you have a massive amount of True Negatives (imbalanced data).
PR Curve (Precision vs Recall): Neither Precision nor Recall use True Negatives in their formulas. Therefore, the PR Curve evaluates the model strictly on how it handles the Positive (minority) class, making it the absolute gold standard for imbalanced classification.

15. If you had unlimited time and resources, how would you improve your models?

Better Preprocessing: Instead of dropping NaN values, I would use a KNNImputer to accurately predict missing data based on similar customers. I would cap extreme outliers (Winsorization) rather than deleting them.
Better Features: I would pull in external APIs (weather data, local city holidays, competitor pricing)

to attach macroeconomic factors to the dataset.
Better Models: I would deploy Deep Learning, utilizing TabNet (Transformers for tabular data) for classification, and state-of-the-art temporal models like LSTMs or Temporal Fusion Transformers for the time-series forecasting. I would use Bayesian Optimization (Optuna) to test millions of hyperparameter combinations automatically over several days.
Better Metrics: I would write a custom "Business Value Loss Function." Instead of mathematically penalizing False Positives and False Negatives equally, the loss function would calculate the exact dollar amount a False Negative costs the hotel (e.g., an empty $300 room) versus a False Positive (e.g., a $50 overbooking penalty), forcing the model to optimize for maximum net revenue rather than standard accuracy.